

PYTHON PROGRAMLAMA DÖNEMİ ÖDEVİ

Akıllı Müşteri Yönetim ve Analiz Sistemi
Telco Senaryosu | Google Colab

1. KISIM: Veri Yapıları ve Temel Mantık

Görev 1: Değişkenler ve Tipler

Bir müşterinin ad_soyad (string), aylık_ücret (float), sadakat_ay (int) ve aktif_mi (boolean) bilgilerini içeren değişkenler tanımlanmıştır. Python'da her değişkenin bir tipi vardır; bu tipler veri üzerinde yapılacak işlemleri belirler.

```
ad_soyad = "Ahmet Yılmaz" # String - metin verisi
aylik_ucret = 650.0 # Float - ondalıklı sayı
sadakat_ay = 30 # Integer - tam sayı
aktif_mi = True # Boolean - True/False
```

Görev 2: Liste ve Sözlük Kullanımı

Şirketin sunduğu 5 hizmet bir Listeye (sıralanmış, değiştirilebilir yapı), müşterinin tüm bilgileri ise bir Sözlüğe (anahtar-değer çifti) kaydedilmiştir.

```
hizmetler_listesi = ["Fiber Internet", "Dijital TV", "Mobil Hat", "Bulut Depolama", "Siber Güvenlik"]

musteri = {
    "ad_soyad": "Ahmet Yılmaz",
    "aylik_ucret": 650.0,
    "sadakat_ay": 30,
    "aktif_mi": True,
    "hizmetler": hizmetler_listesi
}
```

Görev 3: Koşullu İfadeler (If-Else)

Müşterinin aylık_ücret tutarı 500 TL'den fazlaysa VEYA sadakat_ay 24 aydan uzunsa "VIP Müşteri: İndirim Tanımla" mesajı, aksi halde "Standart Müşteri" mesajı ekrana yazdırılır.

```
if muster_i["aylik_ucret"] > 500 or muster_i["sadakat_ay"] > 24:
    print("VIP Müşteri: İndirim Tanımla")
else:
    print("Standart Müşteri")
# Çıktı: VIP Müşteri: İndirim Tanımla
```

Görev 4: String Operasyonları ve Random ID Üretimi

Musterinin adi .upper() metodu ile tamamen büyük harfe çevrilmiş; random kutuphanesi ile IST-2026-XXXX formatında benzersiz bir müşteri ID'si üretilmiştir.

```
import random
ad_buyuk = musteri["ad_soyad"].upper() # "AHMET YILMAZ"
musteri_id = f"IST-2026-{random.randint(1000,9999)}" # "IST-2026-4823"
musteri["musteri_id"] = musteri_id
```

Kritik Soru Cevabi (1. Kısım)

Soru: Müşteri bilgilerini neden 'Liste' yerine 'Sözlük' içinde saklamayı tercih ettik?

Listelerde verilere indeks numarasıyla erişim sağlanır (musteri[0], musteri[1] gibi). Bu yaklaşım hangi indeksin hangi bilgiyi tuttuğunun hatırlanmasını güçleştirir ve kodun okunabilirliğini düşürür. Sözlükte ise verilere anlamlı bir anahtar ile erişim sağlanır: musteri['ad_soyad'], musteri['aylik_ucret']. Bunun başta gelen avantajları şunlardır: (1) Okunabilirlik artar, kod kendini açıklar; (2) Yeni alan eklemek diğer alanların sırasını bozmaz; (3) Anahtar bilindiğinde erişim $O(1)$ hızındadır; (4) Yanlış indeks sırasından kaynaklı hatalar önlenir. Sonuç olarak farklı tipte ve çok alanlı veri modellemek için sözlük çok daha uygun bir yapıdır.

2. KISIM: Fonksiyonlar, Donguler ve Kutuphaneler

Görev 1: 5 Müşterili Ana Liste

Her biri sözlük yapısındaki 5 farklı müşteriden oluşan musteriler_listesi adlı ana liste oluşturulmuştur. Her müşterinin ad_soyad, aylık_ucret, sadakat_ay, aktif_mi, şehir ve hizmetler alanları bulunmaktadır.

Müşteri Adı	Aylık Ücret	Sadakat Ayı	Aktif Mi
Ahmet Yılmaz	650.0 TL	30 ay	Evet
Fatma Kaya	299.0 TL	6 ay	Evet
Mehmet Demir	520.5 TL	18 ay	Hayır
Zeynep Arslan	180.0 TL	3 ay	Evet
Can Öztürk	890.0 TL	48 ay	Evet

Görev 2: Fonksiyon Tanımlama — tutar_hesapla()

aylık_ucret parametresi alan tutar_hesapla fonksiyonu, ücrete yüzde 20 KDV ekleyerek KDV dahil tutarı return ile geri döndürür.

```
def tutar_hesapla(aylik_ucret): # Fonksiyon tanımlama
    kdv_orani = 0.20
    kdv_tutari = aylik_ucret * kdv_orani
    kdv_dahil_tutar = aylik_ucret + kdv_tutari
    return kdv_dahil_tutar # Return ile döndür

# Örnek: tutar_hesapla(500) -> 600.0
```

Görev 3: Kutuphane Kullanımı — math ve datetime

math.ceil() fonksiyonu ile her fatura tutari yukariya yuvarlanmis, datetime.datetime.now() ile de gunun tarihi fatura ciktisinin altina eklenmistir.

```
import math, datetime

for musteri in musteriler_listesi:
    kdv_dahil = tutar_hesapla(musteri["aylik_ucret"])
    yuvarlanmis = math.ceil(kdv_dahil) # math kutuphanesi

bugun = datetime.datetime.now()
tarih_format = bugün.strftime("%d/%m/%Y %H:%M") # datetime kutuphanesi
```

Gorev 4: Hata Denetimi ve Set Kullanimi

Tum musterilerin hizmetleri tekrarli olarak bir listeye eklenmis, ardindan set() fonksiyonu ile benzersiz hizmet kumesi olusturulmustur. try-except bloklari ile olası hatalar yakalanmistir.

```
tum_hizmetler = []
for m in musteriler_listesi:
    for h in m["hizmetler"]:
        tum_hizmetler.append(h) # Tekrarli liste

benzersiz = set(tum_hizmetler) # Set() ile benzersizler

try:
    deger = int("abc") # Hata olusturma
except ValueError as hata:
    print(f"Hata: {hata}") # try-except ile yakalama
```

Gorev 5: Churn Analizi (AI Denetimli Kod)

Asagidaki kod once yapay zekaya yazdirilmiş, hatalı noktalara mudahale edilmiş ve her satirin hangi konuya ait oldugunu gosteren satirici yorum (#) eklenmistir.

```
def churn_analizi(musteriler): # Fonksiyon Tanimlama
    churn_riskli = [] # Liste Kullanimi
    guvenli = []

    for musteri in musteriler: # Dongu Kullanimi
        risk_puani = 0 # Degisken Tanimlama
        risk_sebebi = []

        if musteri["sadakat_ayi"] < 6: # Kosullu Ifade
            risk_puani += 2 # Aritmetik Islem
            risk_sebebi.append("Dusuk sadakat") # Liste Metodu

        if not musteri["aktif_mi"]: # Boolean Kontrolu
            risk_puani += 3
            risk_sebebi.append("Hesap pasif")

        if musteri["aylik_ucret"] < 200: # Float Karsilastirma
            risk_puani += 1
            risk_sebebi.append("Dusuk ucret")
```

```
if risk_puani >= 2: # Kosullu Ifade – risk esigi
    churn_riskli.append({"ad": musteri["ad_soyad"],
                        "puan": risk_puani})
else:
    guvenli.append(musteri["ad_soyad"])

return churn_riskli, guvenli # Return – Tuple donus
```

Churn Analizi Sonuclari:

Musteri Adi	Risk Durumu	Sebepler
Ahmet Yilmaz	Guvenli	—
Fatma Kaya	Guvenli	—
Mehmet Demir	RISKLI	Hesap Pasif
Zeynep Arslan	RISKLI	Sadakat < 6 ay, Dusuk ucret
Can Ozturk	Guvenli	—

Kritik Soru Cevabi (2. Kisim)

Soru: Dongü (for loop) musteri listesini gezerken, hangi musterilerin Churn riskinde oldugunu nasil tespit ettiniz?

Churn tespiti için agirlikli risk puani mantig■ kullanildi. for dongusu her musteride sirayla uc kriteri kontrol eder: (1) sadakat_ayi < 6 ise +2 puan; musterinin hizmetle henuz baglantisi kurulmamistir, rakibe gecme olas■ligi yuksektir. (2) aktif_mi == False ise +3 puan; musteri zaten hizmeti kullanmiyor, bu en guclu ayrilma sinyavidir. (3) ayluk_ucret < 200 TL ise +1 puan; az hizmet alan musteri ek bir sorunla birlikte kolayca ayrilabilir. Her musterinin risk puani toplanir; 2 veya uzeri puan alan musteri 'Churn Riskli' olarak siniflandirilir. Bu yontem tek kritere degil, birden fazla kriterin bir araya gelmesiyle daha isabetli bir tespit yapar.

Sonuc

Bu odev kapsaminda Python'un temel veri yapilari (degisken, liste, sozluk, set, tuple) ve kontrol yapilari (if-else, for dongusu, try-except) ile math, datetime ve random kutuphanalari kullanilarak gercekci bir Telco senaryo sistemi olusturulmustur. Sistem; VIP musteri tespiti, KDV'li fatura hesaplama, benzersiz hizmet listeleme ve cok kriterli Churn risk analizi islevlerini basariyla yerine getirmektedir.